

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 14 – SORTING



SARAH MAULIDYA NATASYAH

109082530023

KELAS S1IF-13-05

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

1. Selection Sort

Selection Sort adalah algoritma pengurutan dengan mencari nilai ekstrim pada sekumpulan data, yang kemudian akan diletakkan pada posisi yang seharusnya. Algoritma ini melibatkan dua proses pencarian nilai ekstrim dan proses pertukaran dua nilai atau *swap*. Terdapat beberapa cara untuk pencarian ini, diantaranya:

- Pertama, mencari nilai terkecil dalam rentang data yang tersisa.
- Data yang ada di posisi paling kiri pada rentang data akan di tukar dengan data yang tersisa.
- Proses ini diulang sampai hanya tersisa satu data.

Jika array yang diurutkan bertipe data `struct`, maka perlu ditambahkan *field* pada saat proses perbandingan nilai ekstrim, kemudian tipe data dari variabel `t` sama dengan `struct` dari arraynya.

2. Insertion Sort

Insertion Sort adalah pengurutan dengan cara menyisipkan suatu nilai pada posisi yang seharusnya. Pengurutan pada algoritma ini dilakukan dengan cukup memilih suatu nilai tertentu kemudian mencari posisinya secara *sequential search*. Cara mengurutkan array dengan algoritma ini diantaranya:

- Untuk satu data yang belum terurut dan sejumlah data yang sudah diurutkan: Geser data yang sudah terurut (ke kanan), sampai ada satu ruang kosong untuk memasukkan data yang belum terurut ke dalam data yang sudah terurut.
- Proses diulang untuk setiap data yang belum terurut terhadap rangkaian data yang sudah terurut.

Algoritma ini melibatkan dua proses pencarian yaitu pencarian *sequential* dan penyisipan.

Jika array yang akan diurutkan bertipe data `struct`, maka perlu ditambahkan *field* pada saat proses perbandingan dalam pencarian posisi, kemudian tipe data dari variabel **temp** sama dengan `struct` dari arraynya.

B. Guided

1. Program Selection Sort pada Array

```
package main

import "fmt"

func SelectionSortArray(arr *[8]int) {
    var idx_min, i, j int
```

```

        for i = 0; i < len(arr) - 1; i++ { //perulangan luar
            idx_min = i;
            for j = i + 1; j < len(arr); j++ { //perulangan dalam
                if arr[j] < arr[idx_min] { //kondisional
                    idx_min = j
                }
            }
            if idx_min != i { //swap
                arr[i], arr[idx_min] = arr[idx_min], arr[i]
            }
        }
    }

func main() {
    var arrAngka [8]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka indeks ke-%d: ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
    fmt.Println()

    SelectionSortArray(&arrAngka)
    fmt.Println("=== SETELAH SORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " ")
    }
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Guided\Guided-1\Guided-1.go"
Masukkan data angka indeks ke-0: 3
Masukkan data angka indeks ke-1: 7
Masukkan data angka indeks ke-2: 4
Masukkan data angka indeks ke-3: 1
Masukkan data angka indeks ke-4: 8
Masukkan data angka indeks ke-5: 9
Masukkan data angka indeks ke-6: 4
Masukkan data angka indeks ke-7: 2

=== SEBELUM SORTING ===
3 7 4 1 8 9 4 2
=== SETELAH SORTING ===
1 2 3 4 4 7 8 9
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah>

```

2. Program Selection Sort pada Struct

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(sMHS *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(sMHS) - 1; i++ { //loop luar
        idx_min = i
        for j = i + 1; j < len(sMHS); j++ { //loop dalam
            mencari nilai ekstrim
            if sMHS[j].IPK < sMHS[idx_min].IPK { //sorting
                terkecil ke terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
        }
    }
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        i)
        fmt.Printf("Nama: ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Printf("NIM: ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Printf("IPK: ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORTING ===")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
        fmt.Printf("Nama: %s\n", structMHS[i].nama)
        fmt.Printf("NIM: %s\n", structMHS[i].nim)
        fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}
```

```

        SelectionSortArray(&structMHS)
        fmt.Println("=== SETELAH SORTING ===")
        for i := 0; i < len(structMHS); i++ {
            fmt.Printf("--- Data Mahasiswa ke-%d ---\n", i)
            fmt.Printf("Nama: %s\n", structMHS[i].nama)
            fmt.Printf("NIM: %s\n", structMHS[i].nim)
            fmt.Printf("IPK: %.2f\n", structMHS[i].IPK)
        }
        fmt.Println("=====")
        fmt.Println()
    }
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Guided\Guided-2\Guided-2.go"
--- Masukkan Data Mahasiswa ke-0 ---
Nama: dhimas
NIM: 123
IPK: 3.21
--- Masukkan Data Mahasiswa ke-1 ---
Nama: hafizh
NIM: 234
IPK: 2.46
--- Masukkan Data Mahasiswa ke-2 ---
Nama: fathur
NIM: 345
IPK: 3.58
--- Masukkan Data Mahasiswa ke-3 ---
Nama: rahman
NIM: 456
IPK: 1.57
--- Masukkan Data Mahasiswa ke-4 ---
Nama: masdim
NIM: 567
IPK: 3.96

```

```

=== SEBELUM SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: dhimas
NIM: 123
IPK: 3.21
--- Data Mahasiswa ke-1 ---
Nama: hafizh
NIM: 234
IPK: 2.46
--- Data Mahasiswa ke-2 ---
Nama: fathur
NIM: 345
IPK: 3.58

```

```

--- Data Mahasiswa ke-3 ---
Nama: rahman
NIM: 456
IPK: 1.57
--- Data Mahasiswa ke-4 ---
Nama: masdim
NIM: 567
IPK: 3.96
=====

=== SETELAH SORTING ===
--- Data Mahasiswa ke-0 ---
Nama: rahman
NIM: 456
IPK: 1.57
--- Data Mahasiswa ke-1 ---
Nama: hafizh
NIM: 234
IPK: 2.46
--- Data Mahasiswa ke-2 ---
Nama: dhimas
NIM: 123
IPK: 3.21
--- Data Mahasiswa ke-3 ---
Nama: fathur
NIM: 345
IPK: 3.58
--- Data Mahasiswa ke-4 ---
Nama: masdim
NIM: 567
IPK: 3.96
=====

```

PS D:\ALPRO 2\109082530023 Sarah-Maulidya-Natasyah> █

3. Program *Insertion Sort* pada Array Descending

```

package main

import "fmt"

type arrInt [100]int

func insertionSort(T *arrInt, n int) {
    var temp, i, j int
    for i = 1; i <= n - 1; i++ {
        temp = T[i]
        j = i
        for j > 0 && temp > T[j - 1] {
            T[j] = T[j - 1]
            j--
        }
        T[j] = temp
    }
}

```

```

    }
}

func main() {
    var data arrInt
    var n, i int

    fmt.Println("Masukkan jumlah data: ")
    fmt.Scan(&n)

    fmt.Println("Masukkan data: ")

    for i = 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    insertionSort(&data, n)

    fmt.Println("\n\nData setelah sorting (Descending): ")
    for i = 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }

    fmt.Println()
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Guided\Guided-3\Guided-3.go"
Masukkan jumlah data: 5
Masukkan data:
7 3 9 1 5

Data sebelum sorting:
7 3 9 1 5

Data setelah sorting (Descending):
9 7 5 3 1
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> 

```

4. Program *Insertion Sort* pada Struct Mahasiswa

```

package main

import "fmt"

```

```

type mahasiswa struct {
    nama string
    nim  string
    ipk  float64
}

type arrMhs [100]mahasiswa

func insertionSortStruct(T *arrMhs, n int) {
    var temp mahasiswa
    var i, j int

    for i = 1; i < n; i++ {
        temp = T[i]
        j = i - 1
        for j >= 0 && T[j].nama < temp.nama {
            T[j + 1] = T[j]
            j--
        }
        T[j + 1] = temp
    }
}

func main() {
    var data arrMhs
    var n, i int

    fmt.Print("Masukkan jumlah mahasiswa: ")
    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Println("\nData mahasiswa ke-", i + 1)

        fmt.Print("Nama: ")
        fmt.Scan(&data[i].nama)

        fmt.Print("NIM: ")
        fmt.Scan(&data[i].nim)

        fmt.Print("IPK: ")
        fmt.Scan(&data[i].ipk)
    }

    fmt.Println("\nData sebelum sorting: ")
    for i = 0; i < n; i++ {
        fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
    }

    insertionSortStruct(&data, n)
}

```



```

        fmt.Println("\nData setelah sorting (Descending
berdasarkan Nama): ")
        for i = 0; i < n; i++ {
            fmt.Println(data[i].nama, data[i].nim, data[i].ipk)
        }
    }
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Guided\Guided-4\Guided-4.go"
Masukkan jumlah mahasiswa: 3

Data mahasiswa ke- 1
Nama: rere
NIM: 101
IPK: 4

Data mahasiswa ke- 2
Nama: regita
NIM: 102
IPK: 4

Data mahasiswa ke- 3
Nama: reguta
NIM: 103
IPK: 4

Data sebelum sorting:
rere 101 4
regita 102 4
reguta 103 4

Data setelah sorting (Descending berdasarkan Nama):
rere 101 4
reguta 103 4
regita 102 4
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> 

```

C. Unguided

1. Soal Unguided 1 Selection Sort

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu.

Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program **rumahkerabat** yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma *selection sort*.

Masukan dimulai dengan sebuah integer **n** ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi **n** baris berikutnya selalu dimulai dengan sebuah integer **m** ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di

daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar di masing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban:

```
package main

import "fmt"

const max = 1000000

type array [max]int

func selectionSort(arr *array, n int) {
    var idx_min, temp int

    for i := 0; i < n - 1; i++ {
        idx_min = i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        temp = arr[idx_min]
        arr[idx_min] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var data array
        for j := 0; j < m; j++ {
            var no int
```

```

        fmt.Scan(&no)
        data[j] = no
    }

    selectionSort(&data, m)

    for j := 0; j < m; j++ {
        if j > 0 {
            fmt.Print(" ")
        }
        fmt.Print(data[j])
    }
    fmt.Println()
}
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Unguided\Unguided-1\Unguided-1.go"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> 

```

Penjelasan:

Program ini digunakan untuk menyusun nomor rumah kerabat Hercules di setiap daerah secara ascending menggunakan *Selection Sort*. Fungsi `selectionSort` digunakan untuk mencari elemen terkecil pada array yang belum terurut dan menyimpan indeksinya di variabel `idx_min`, kemudian nilai tersebut ditukar ke posisi terdepan menggunakan variabel sementara `temp`. Pada fungsi `main`, program akan membaca input `n` sebagai jumlah daerah, kemudian setiap daerah akan membaca `m` nomor rumah ke dalam array dan mengurutkannya menggunakan fungsi `selectionSort`. Output dicetak dalam satu baris per daerah.

2. Soal Unguided 2 Selection Sort

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program **kerabat dekat** yang akan menampilkan nomor rumah mulai dari nomor yang ganjil lebih dulu terurut

membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, dimasing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban:

```
package main

import "fmt"

func selectionSortA(arr []int, n int) {
    var idx_min, temp int

    for i := 0; i < n-1; i++ {
        idx_min = i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idx_min] {
                idx_min = j
            }
        }
        temp = arr[idx_min]
        arr[idx_min] = arr[i]
        arr[i] = temp
    }
}

func SelectionSortD(arr []int, n int) {
    var idx_max, temp int
```

```

    for i := 0; i < n-1; i++ {
        idx_max = i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[idx_max] {
                idx_max = j
            }
        }
        temp = arr[idx_max]
        arr[idx_max] = arr[i]
        arr[i] = temp
    }
}

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        var ganjil []int
        var genap []int

        for j := 0; j < m; j++ {
            var no int
            fmt.Scan(&no)

            if no%2 != 0 {
                ganjil = append(ganjil, no)
            } else {
                genap = append(genap, no)
            }
        }

        selectionSortA(ganjil, len(ganjil))
        SelectionSortD(genap, len(genap))

        first := true

        for j := 0; j < len(ganjil); j++ {
            if !first {
                fmt.Print(" ")
            }
            fmt.Print(ganjil[j])
            first = false
        }

        for j := 0; j < len(genap); j++ {
            if !first {

```

```

        fmt.Print(" ")
    }

    fmt.Print(genap[j])
    first = false
}
fmt.Println()
}
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Unguided\Unguided-2\Unguided-2.go"
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> 

```

Penjelasan:

Program ini digunakan untuk menyusun nomor rumah kerabat Hercules di setiap daerah. Nomor rumah ganjil akan ditampilkan lebih dulu secara *ascending*, kemudian rumah dengan nomor genap akan ditampilkan secara *descending*. Fungsi `selectionSortA` digunakan untuk mengurutkan *slice* dari kecil ke besar dengan mencari nilai terkecil menggunakan `idx_min` sedangkan fungsi `selectionSortD` digunakan untuk mengurutkan nilai dari besar ke kecil dengan mencari nilai terbesar menggunakan `idx_max`, keduanya menggunakan `temp` sebagai variabel sementara. Pada fungsi `main`, program akan membaca input dari pengguna berupa `n` sebagai jumlah daerah, lalu setiap nomor rumah langsung dipisahkan ke dalam dua *slice* yaitu ganjil dan genap berdasarkan hasil dari modulo. Output dicetak dalam satu baris per daerah menggunakan variabel `first` untuk menghindari spasi yang berlebihan.

3. Soal Unguided 3 Insertion Sort

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda *insertion sort*), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan ke dalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris kedua adalah status jarak setiap bilangan yang ada di dalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran:

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
2	4 40 14 8 26 1 38 2 32 -31	1 2 4 8 14 26 32 38 40 Data berjarak tidak tetap

Jawaban:

```
package main

import "fmt"

func insertionSort(arr []int, n int) {
    var temp int

    for i := 1; i < n; i++ {
        temp = arr[i]
        j := i
        for j > 0 && temp < arr[j - 1] {
            arr[j] = arr[j - 1]
            j--
        }
        arr[j] = temp
    }
}

func main() {
    var arr []int
    var x int

    for {
        fmt.Scan(&x)

        if x < 0 {
            break
        }

        arr = append(arr, x)
    }

    n := len(arr)

    if n > 0 {
        insertionSort(arr, n)
    }
}
```

```

    for i := 0; i < n; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(arr[i])
    }
    fmt.Println()

    if n < 2 {
        fmt.Println("Data berjarak tidak tetap")
    } else {
        jarak := arr[1] - arr[0]
        tetap := true

        for i := 1; i < n - 1; i++ {
            if arr[i + 1] - arr[i] != jarak {
                tetap = false
                break
            }
        }

        if tetap {
            fmt.Printf("Data berjarak %d\n", jarak)
        } else {
            fmt.Println("Data berjarak tidak tetap")
        }
    }
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Unguided\Unguided-3\Unguided-3.go"
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Unguided\Unguided-3\Unguided-3.go"
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah>

```

Penjelasan:

Program ini digunakan untuk membaca sekumpulan bilangan bulat kemudian mengurutkannya menggunakan *Insertion Sort*, kemudian memeriksa apakah jarak antar bilangan yang terurut selalu sama atau tidak. Fungsi `insertionSort` digunakan untuk menyisipkan setiap elemen ke posisi yang tepat dengan cara menyimpannya ke variabel sementara (`temp`) lalu menggeser elemen yang lebih

besar ke kanan sampai posisi yang sesuai ditemukan. Pada fungsi `main`, bilangan akan dibaca satu per satu dan hanya bilangan yang tidak negatif disimpan ke dalam array. Setelah diurutkan, selisih antar elemen dibandingkan satu per satu terhadap jarak pertama, lalu output yang ditampilkan “Data berjarak x” jika seluruh jaraknya sama, atau jika tidak maka output yang ditampilkan adalah “Data berjarak tidak tetap”.

4. Soal *Unguided* 4 Insertion Sort

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau *field* pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n :
integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data
buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n :
integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
terfavorit, yaitu memiliki rating tertinggi}
```

```

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer
)
{I.S. Array pustaka berisi n data buku
F.S. Array pustaka terurut menurun/mengecil terhadap rating.
Catatan: Gunakan metoda Insertion sort}

procedure Cetak5Terbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut
rating
F.S. Laporan 5 judul buku dengan rating tertinggi
Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r :
integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut
rating
F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
eksemplar, rating) dengan rating yang diberikan. Jika tidak
ada buku dengan rating yang ditanyakan, cukup tuliskan "Tidak
ada buku dengan rating seperti itu". Catatan: Gunakan
pencarian biner/belah dua.}

```

Jawaban:

```

package main

import "fmt"

const nMax = 7919

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

type DaftarBuku [nMax]Buku

func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Scan(&pustaka[i].id, &pustaka[i].judul,
&pustaka[i].penulis, &pustaka[i].penerbit,
&pustaka[i].eksemplar, &pustaka[i].tahun,
&pustaka[i].rating)
    }
}

func CetakTerFavorit(pustaka DaftarBuku, n int) {
    idx_max := 0
    for i := 1; i < n; i++ {

```

```

        if pustaka[i].rating > pustaka[idx_max].rating {
            idx_max = i
        }
    }
    fmt.Println(pustaka[idx_max].judul,
pustaka[idx_max].penulis, pustaka[idx_max].penerbit,
pustaka[idx_max].tahun)
}

func UrutBuku(pustaka *DaftarBuku, n int) {
    var temp Buku

    for i := 1; i <= n-1; i++ {
        j := i
        temp = pustaka[j]
        for j > 0 && temp.rating > pustaka[j-1].rating {
            pustaka[j] = pustaka[j-1]
            j--
        }
        pustaka[j] = temp
    }
}

func Cetak5Terbaru(pustaka DaftarBuku, n int) {
    var batas int

    batas = 5
    if n < 5 {
        batas = n
    }

    for i := 0; i < batas; i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(pustaka[i].judul)
    }
    fmt.Println()
}

func CariBuku(pustaka DaftarBuku, n int, r int) {
    var left, right, found int

    left = 0
    right = n - 1
    found = -1

    for left <= right {
        mid := (left + right) / 2
        if pustaka[mid].rating == r {
            found = mid
        }
    }
}

```

```

        break
    } else if pustaka[mid].rating > r {
        left = mid + 1
    } else {
        right = mid - 1
    }
}

if found != -1 {
    fmt.Println(pustaka[found].judul,
pustaka[found].penulis, pustaka[found].penerbit,
pustaka[found].tahun, pustaka[found].rating)
} else {
    fmt.Println("Tidak ada buku dengan rating seperti
itu")
}
}

func main() {
    var n int
    var pustaka DaftarBuku

    fmt.Scan(&n)
    DaftarkanBuku(&pustaka, n)
    CetakTerFavorit(pustaka, n)
    UrutBuku(&pustaka, n)
    Cetak5Terbaru(pustaka, n)

    var cariRating int
    fmt.Scan(&cariRating)
    CariBuku(pustaka, n, cariRating)
}

```

Output:

```

PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> go run "d:\ALPRO 2\109082530023_
Sarah-Maulidya-Natasyah\Week-12_Modul-14\Unguided\Unguided-4\Unguided-4.go"
5
B1 Laskar_Pelangi Andrea_Hirata Bentang 10 2005 4
B2 Bumi_Manusia Pramoedya Hasta_Mitra 5 1980 5
B3 Laut_Bercerita Leila_S.K. Gramedia 8 2017 4
B4 Filosofi_Teras Henry_Manampiring Kompas 12 2018 5
B5 Dilan Pidi_Baiq Mizan 20 2014 3
Bumi_Manusia Pramoedya Hasta_Mitra 1980
Bumi_Manusia Filosofi_Teras Laskar_Pelangi Laut_Bercerita Dilan
4
Laskar_Pelangi Andrea_Hirata Bentang 2005 4
PS D:\ALPRO 2\109082530023_Sarah-Maulidya-Natasyah> 

```

Penjelasan:

Program ini digunakan untuk mengelola data buku di perpustakaan, mulai dari membaca data, mengurutkan, hingga mencari buku berdasarkan rating. Fungsi `DaftarkanBuku` digunakan untuk membaca seluruh data buku dan menyimpannya ke dalam array. Fungsi `CetakTerFavorit` digunakan untuk mencari dan menampilkan buku dengan rating tertinggi. Fungsi `UrutBuku` digunakan untuk mengurutkan data buku secara *descending* berdasarkan rating menggunakan *Insertion Sort* menggunakan variabel sementara `temp`. Fungsi `Cetak5Terbaru` digunakan untuk menampilkan lima judul buku dengan rating tertinggi, dengan mempertimbangkan kemungkinan data kurang dari lima. Fungsi `CariBuku` digunakan untuk mencari buku berdasarkan rating menggunakan *binary search*, dan menampilkan datanya jika ditemukan atau jika tidak ditemukan akan mencetak pesan “Tidak ada buku dengan rating seperti itu”. Pada fungsi `main`, program akan menampilkan hasil dari seluruh fungsi secara berurutan.

D. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa *selection sort* digunakan untuk mengurutkan data dengan cara mencari nilai ekstrim (terkecil atau terbesar) lalu menukarnya ke posisi yang tepat, sedangkan *insertion sort* digunakan dengan menyisipkan elemen ke dalam posisi yang benar pada bagian data yang sudah terurut. Pada modul ini, *selection sort* dan *insertion sort* digunakan untuk mengelola data perpustakaan, seperti mengurutkan buku berdasarkan rating secara *descending* menggunakan variabel sementara (`temp`) yang bertipe *struct*. Data praktikum ini, dapat dipahami bahwa algoritma sorting sangat penting dalam pengolahan data terstruktur agar informasi dapat ditampilkan secara sistematis, serta mempermudah proses pencarian data di tahap selanjutnya.